

This document provides a guide for initializing a Git repository, establishing a project structure, and configuring GitHub branch protection:

- **Workflow Setup:** Covers the end-to-end process of creating directories, initializing Git, staging, committing, and pushing code to a remote GitHub repository.
- **Branch Protection Troubleshooting:** Details how to handle branch protection limitations on free private repositories by suggesting solutions like making the repository public, upgrading the account, or utilizing CI/CD status checks.
- **Verification and Workflow:** Explains how to test branch protection enforcement, set up pull request workflows, and work around approval restrictions when merging changes.

Git Basics for Beginners

Git is a version control system that helps you track changes to your source code and collaborate effectively with others. It allows you to revert files to previous states, review change history, and coordinate work across teams.

1. Installation and Configuration

Check if Git is already installed on your system by running the following command in your terminal or command prompt:

```
Shell  
git --version
```

If Git is not installed, download and install it from the official website (git-scm.com).
Next, configure your identity so your commits are properly attributed to you:

```
Shell  
git config --global user.name "Your Name"  
git config --global user.email "your_email@example.com"
```

2. Creating a Repository

A Git repository stores all files and revision history for a project. You can either initialize a new repository locally or clone an existing one.

Initializing a New Repository

```
Shell  
mkdir my_project  
cd my_project  
git init
```

Cloning an Existing Repository

Shell

```
git clone https://github.com/username/repository.git
```

3. Basic Git Workflow

Git manages files across four main areas:

- **Working Directory:** The local directory where you directly edit and create files.
- **Staging Area (Index):** A preview area where changes are organized before being committed.
- **Local Repository:** Your local copy of the repository storing all committed versions.
- **Remote Repository:** A server-hosted version (e.g., GitHub) used to collaborate with team members.

Staging and Committing Changes

To add specific modified files or all changes to the staging area:

Shell

```
# Stage a specific file
```

```
git add filename
```

```
# Stage all modified and new files
```

```
git add .
```

Save the staged snapshot to your local repository history using a descriptive commit message:

Shell

```
git commit -m "Add descriptive commit message here"
```

Synchronizing with Remote Repositories

Push your local commits to the remote repository:

Shell

```
git push origin main
```

Fetch and merge changes from the remote repository to update your local copy:

Shell

```
git pull origin main
```

4. Viewing History

To review the chronological log of commits in the repository:

```
Shell  
git log
```

Git Repository Setup & Initial Deployment

Initialize a Git repository, create a directory structure, and push your work to a remote repository using Git Bash, follow these steps:

1. Create and Enter Directory

Start by creating your project folder and navigating into it:

```
mkdir -p databricks-enterprise-training  
cd databricks-enterprise-training
```

```
e440624@SGALAPTOP1046 MINGW64 ~  
$ cd databricks-enterprise-training
```

```
Shell  
mkdir -p databricks-enterprise-training  
cd databricks-enterprise-training
```

2. Initialize Git

Turn the current directory into a Git repository:

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)  
$ git init
```

```
Shell  
git init
```

3. Establish Project Structure

Create the necessary subdirectories for your project (e.g., config, utils, and source folders):

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)
$ mkdir -p config/targets \
    utils \
    src/01_governance \
    src/02_bronze \
    src/03_silver \
    src/04_gold \
    src/05_deployment
```

Shell

```
mkdir -p config/targets \
    utils \
    src/01_governance \
    src/02_bronze \
    src/03_silver \
    src/04_gold \
    src/05_deployment
```

4. Create Placeholder Files

Use the touch command to create empty files where your code or configuration will go, ensuring the structure is ready:

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)
$ touch databricks.yml README.md \
    config/targets/dev.yml \
    config/targets/prod.yml \
    utils/stream_simulator.py \
    src/01_governance/unity_catalog_security.sql \
    src/02_bronze/ingest_raw_events.py \
    src/03_silver/cleanse_dlt_pipeline.py \
    src/04_gold/aggregate_metrics.py \
    src/04_gold/backup_recovery.py \
    src/05_deployment/ci_cd_pipeline.py
```

Shell

```
touch databricks.yml README.md \
    config/targets/dev.yml \
    config/targets/prod.yml \
    utils/stream_simulator.py \
    src/01_governance/unity_catalog_security.sql \
    src/02_bronze/ingest_raw_events.py \
    src/03_silver/cleanse_dlt_pipeline.py \
    src/04_gold/aggregate_metrics.py \
```

```
src/04_gold/backup_recovery.py \  
src/05_deployment/ci_cd_pipeline.py
```

Verify the Structure

Confirm your directory tree looks correct before committing:

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)  
$ find . -maxdepth 3 -not -path '*/.*'  
.  
./config  
./config/targets  
./config/targets/dev.yml  
./config/targets/prod.yml  
./databricks.yml  
./README.md  
./src  
./src/01_governance  
./src/01_governance/unity_catalog_security.sql  
./src/02_bronze  
./src/02_bronze/ingest_raw_events.py  
./src/03_silver  
./src/03_silver/cleanse_dlt_pipeline.py  
./src/04_gold  
./src/04_gold/aggregate_metrics.py  
./src/04_gold/backup_recovery.py  
./src/05_deployment  
./src/05_deployment/ci_cd_pipeline.py  
./utils  
./utils/stream_simulator.py
```

Shell

```
find . -maxdepth 3 -not -path '*/.*'
```

5. Stage and Commit

Add all files to the staging area and create your initial commit:

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)  
$ git add .
```

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)  
$ git commit -m "feat: initialize repository directory structure and bundle layout"  
[main 8a08ec0] feat: initialize repository directory structure and bundle layout  
Committer: Diksha Soni <diksha.soni@sganalytics.com>  
Your name and email address were configured automatically based  
on your username and hostname. Please check that they are accurate.  
You can suppress this message by setting them explicitly. Run the  
following command and follow the instructions in your editor to edit  
your configuration file:  
  
    git config --global --edit  
  
After doing this, you may fix the identity used for this commit with:  
  
    git commit --amend --reset-author  
  
11 files changed, 0 insertions(+), 0 deletions(-)  
create mode 100644 README.md  
create mode 100644 config/targets/dev.yml  
create mode 100644 config/targets/prod.yml  
create mode 100644 databricks.yml  
create mode 100644 src/01_governance/unity_catalog_security.sql  
create mode 100644 src/02_bronze/ingest_raw_events.py  
create mode 100644 src/03_silver/cleanse_dlt_pipeline.py  
create mode 100644 src/04_gold/aggregate_metrics.py  
create mode 100644 src/04_gold/backup_recovery.py  
create mode 100644 src/05_deployment/ci_cd_pipeline.py  
create mode 100644 utils/stream_simulator.py
```

Shell

```
git add .
```

```
git commit -m "feat: initialize repository directory structure  
and bundle layout"
```

6. Set Branch and Remote

Ensure your default branch is named **main** and link your local repository to your remote GitHub repository:

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)  
$ git branch -M main
```

Shell

```
git branch -M main
```

7. Link Remote Repository

Link your GitHub remote repository (Replace with your actual repo URL):

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)  
$ git remote add origin https://github.com/sga-databricks-lab/databricks-enterprise-training.git
```

Shell

```
git remote add origin  
https://github.com/sga-databricks-lab/databricks-enterprise-training.git
```

8. Push to Remote

Finally, upload your local changes to the remote main branch:

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)  
$ git push -u origin main  
Enumerating objects: 15, done.  
Counting objects: 100% (13/13), done.  
Delta compression using up to 12 threads  
Compressing objects: 100% (9/9), done.  
Writing objects: 100% (11/11), 1.10 KiB | 280.00 KiB/s, done.  
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
To https://github.com/sga-databricks-lab/databricks-enterprise-training.git  
 3043ab7..8a08ec0  main -> main  
branch 'main' set up to track 'origin/main'.
```

Shell

```
git push -u origin main
```

9. Clean Up Temporary Files

If you need to clean up temporary files like `.gitkeep` later, you can remove them using `git rm` (e.g., `git rm config/.gitkeep` or `find config utils src -name ".gitkeep" -exec git rm {} +`) and commit those changes afterward.

```
git rm config/.gitkeep
```

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)
$ find config utils src -name ".gitkeep" -exec git rm {} +
rm 'config/.gitkeep'
rm 'src/.gitkeep'
rm 'utils/.gitkeep'
```

Shell

```
git rm config/.gitkeep
find config utils src -name ".gitkeep" -exec git rm {} +
```

10. Push into main origin

```
e440624@SGALAPTOP1046 MINGW64 ~/databricks-enterprise-training (main)
$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 515 bytes | 257.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/sga-databricks-lab/databricks-enterprise-training.git
 8a08ec0..5c89241  main -> main
```

Shell

```
git push origin main
```

By default, GitHub does not protect the main branch automatically—anyone with write access can push code directly to it.

To enable branch protection with your exact requirements, follow these steps in your repository settings:

Step 1: Open Branch Protection Settings

1. Go to your repository on GitHub.
2. Click on **Settings** in the top navigation bar.
3. In the left sidebar, click **Branches**.
4. Click **Add branch protection rule** (or **Add rule**).

Step 2: Apply Your Protection Checks

- **Branch name pattern:** Enter `main`.
- Check **Require a pull request before merging:**
 - Set **Required approvals** to `1`.
 - Check **Dismiss stale pull request approvals when new commits are pushed**.
 - Check **Require approval of the most recent reviewable push**.
- Check **Require status checks to pass before merging:**
 - Check **Require branches to be up to date before merging**.
 - Select your repository's active CI/CD checks from the list.
- Check **Require conversation resolution before merging**.
- Check **Do not allow bypassing the above settings** (or **Include administrators** depending on your GitHub plan). This prevents admin overrides.
- Ensure **Allow force pushes** remain **unchecked** (force pushes are blocked by default once protection is enabled).

Step 3: Save Changes

Click **Create** or **Save changes** at the bottom of the page.

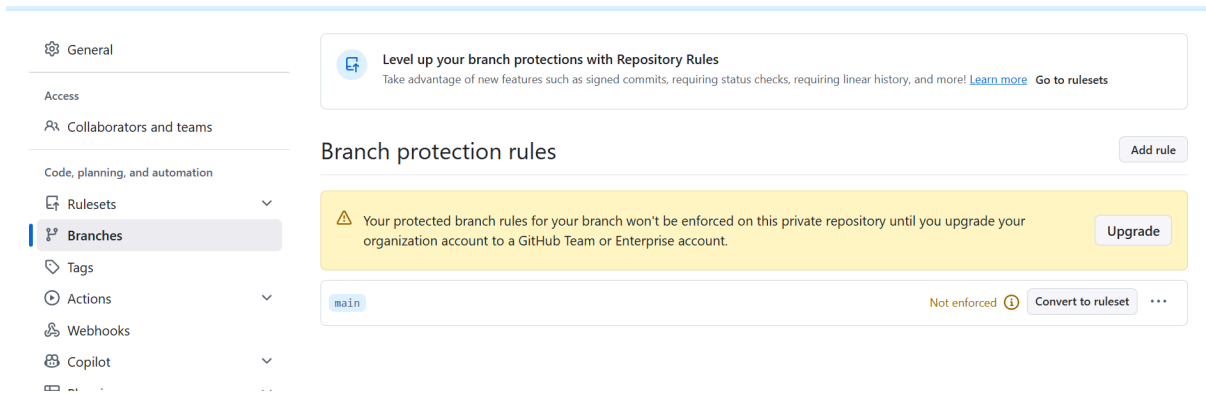
Confirm access



Signed in as @dishu5859

Enter the verification code sent to d*****@sganalytics.com. If it doesn't appear within a few minutes, check your spam folder.

Verify



Option 1: Make the Repository Public (Free Solution)

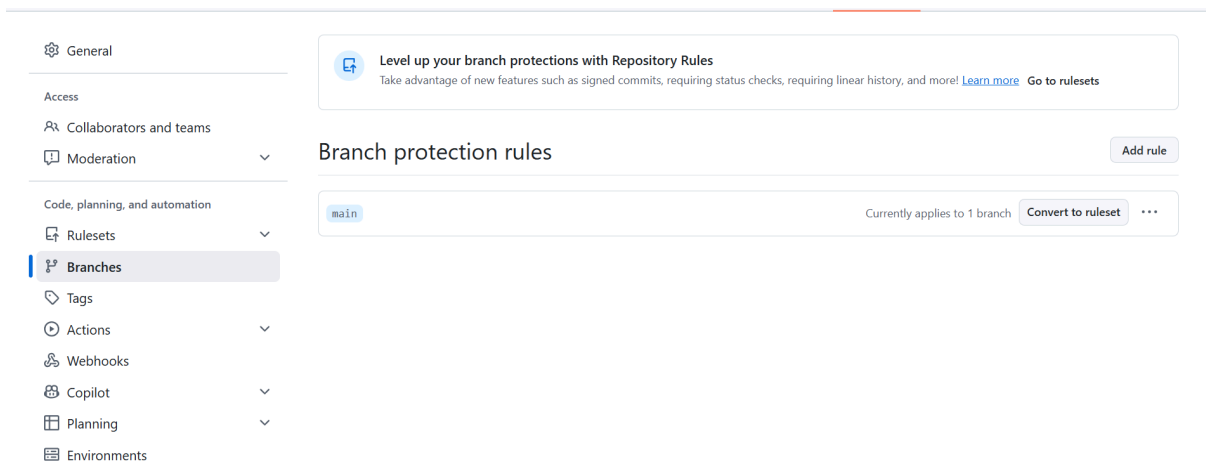
- If the code is not confidential, change the repository visibility to Public.
- On public repositories, all Branch Protection Rules are enforced completely free, even on the GitHub Free plan.
- **Steps:** Repo Settings -> General -> scroll down to Danger Zone -> Change visibility -> Make public.

Option 2: Upgrade the Account (Paid Solution)

- If keeping the repository private is essential, you will need to upgrade your organization account to a GitHub Team or Enterprise plan (by clicking the Upgrade button shown in the screenshot).

Option 3: GitHub Actions CI/CD (Free Workaround for Private Repos)

- Without rules enforcement, direct pushes won't be hard-blocked, but you can enforce status checks using a GitHub Actions workflow so that PRs cannot be merged if checks fail.



Your Branch Protection rule has been successfully activated (it shows "Currently applies to 1 branch" and the warning has disappeared).

To verify, test the following:

- **Direct Push Test:** Try to push directly to the `main` branch in your local terminal (`git push origin main`). GitHub will reject it: `GH006: Protected branch hook declined`.

Shell

```
git push origin main
```

- **Pull Request Workflow:** Now, to push any changes, you will need to create a feature branch, open a pull request, get 1 review approval, and all conversations must be resolved before code can be merged into `main`.

Step-by-step Approval Process:

sga-databricks-lab / databricks-enterprise-training

Filters Labels 10 Milestones 0 New pull request

2 Open 2 Closed

Feature/ingest script
#4 opened 5 hours ago by dishu5859 (Member) • Review required

Update README.md
#1 opened 2 days ago by dishu5859 (Member) • Review required

ProTip! Notify someone on an issue with a mention, like: @dishu5859.

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

- **Open the PR:** Click on a PR from the list (for example, `Update README.md #1`).
- **Go to the Files Changed Tab:** Click on the **Files changed** tab at the top of the page.

sga-databricks-lab / databricks-enterprise-training

Feature/ingest script #4

Awaiting approval Code

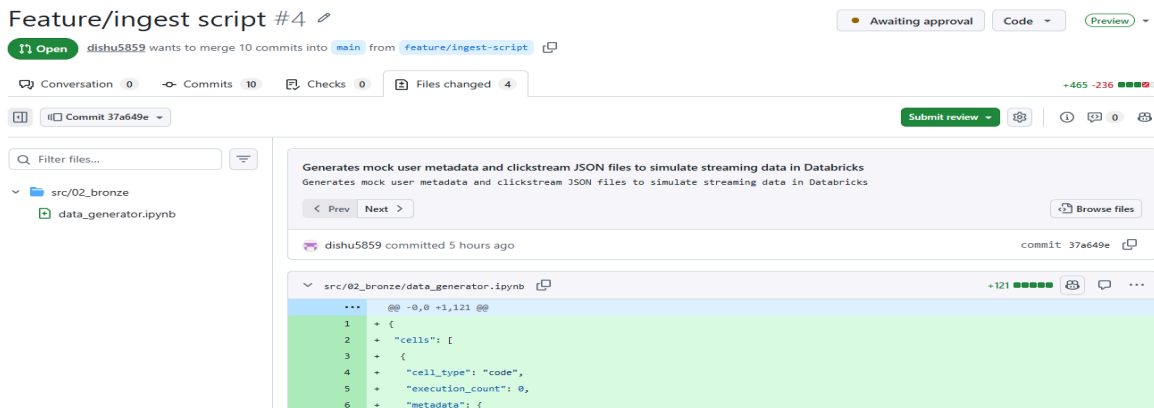
Open dishu5859 wants to merge 10 commits into `main` from `feature/ingest-script`

Conversation Commits Checks Files changed

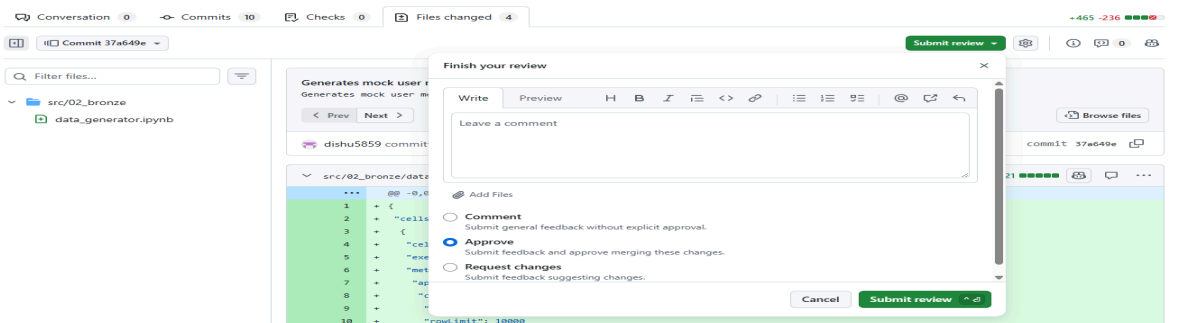
Commits on Sep 3, 2025

File	Author	Commit
Generates mock user metadata and clickstream JSON files to simulate streaming data in Databricks	dishu5859	37ab09e
Auto Loader stream ingestion script	dishu5859	4b5b59f
Auto Loader stream ingestion script	dishu5859	a28855e
Auto Loader stream ingestion script	dishu5859	78981c1
Auto Loader stream ingestion script	dishu5859	9388e62

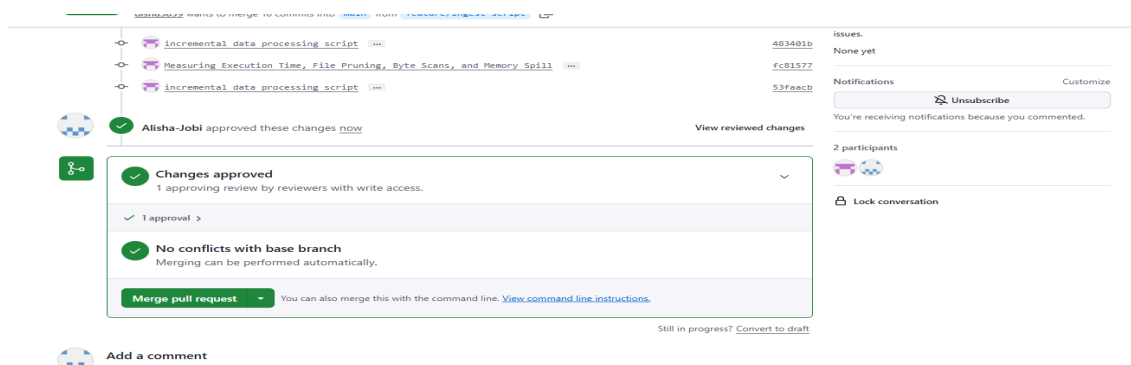
- **Click the Review Changes Button:** Click the green **Review changes** button located in the upper right corner.

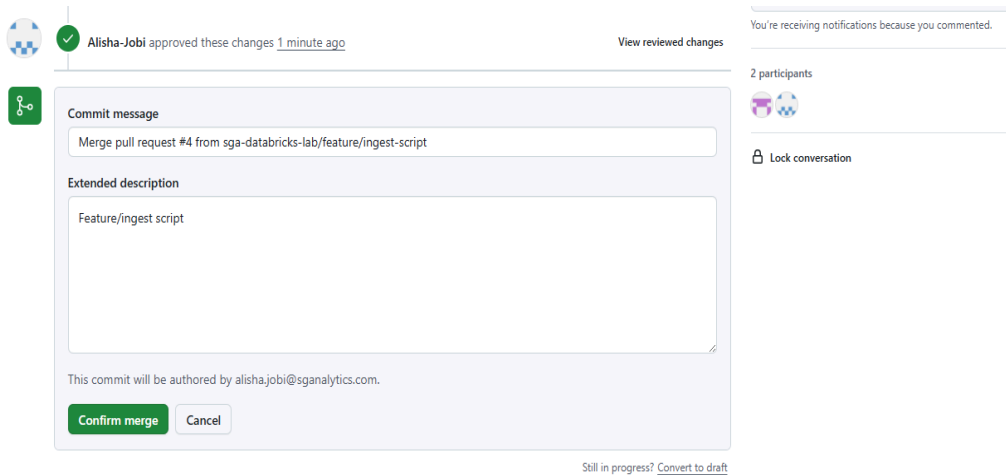


- **Select Approve:** In the dropdown menu that appears:
 - Select the **Approve** radio button.
 - Click the green **Submit review** button.



- **Select Merger Pull Request:** In the dropdown menu that appears:
 - Select the **Approve** radio button.
 - Click the green **Merger Pull request** button.
 - Confirm Merger.

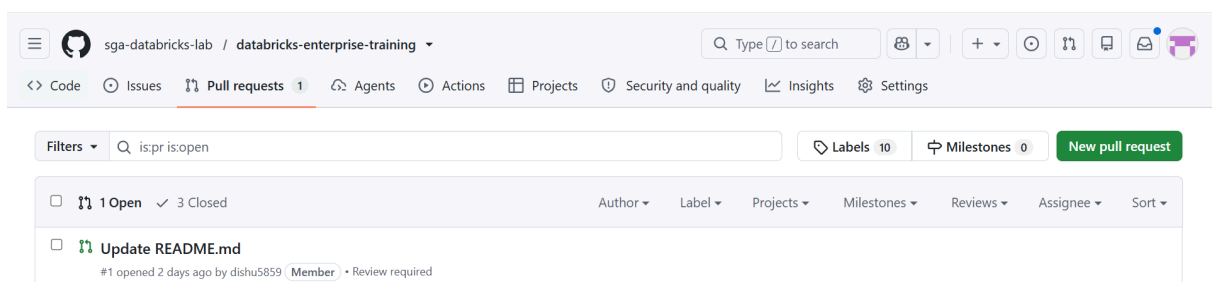




Important Note (If you are currently logged in as dishu5859):

Both PRs in the screenshot were created by **dishu5859**. GitHub does not allow you to approve your own pull request.

- **To have a teammate approve it:** Ask a teammate to log in to their own GitHub account and follow the steps above.
- **If you are working alone and need to merge immediately:**
 1. Go to Repo **Settings** -> **Branches**.
 2. Click **Edit** next to the **main** branch rule.
 3. Temporarily uncheck **"Require a pull request before merging"** (or set Required approvals to **0**) and click **Save changes**.
 4. After this, the red warning on the PR page will disappear, allowing you to directly click the **Merge pull request** button.
- Once the merger request is approved, remove the file from the pull request and move all files into the main folder.



- After approved and merger you can now see all the files visible in the bronze layer.

sga-databricks-lab / databricks-enterprise-training

Q Type [1] to search

+

<> Code

Issues

Pull requests 1

Agents

Actions

Projects

Security and quality

Insights

Settings

Files

main

+

Q

Q Go to file

T

> config

> src

> 01_governance

> 02_bronze

data_generator.ipynb

generate_new_batch.ipynb

ingest_raw_events.ipynb

databricks-enterprise-training / src / 02_bronze /

Add file

...

dishu5859 incremental data processing script

53faacb · 3 hours ago

History

Name	Last commit message	Last commit date
..		
data_generator.ipynb	Generates mock user metadata and clickstream JSON files to s...	6 hours ago
generate_new_batch.ipynb	incremental data processing script	3 hours ago
ingest_raw_events.ipynb	Auto Loader stream ingestion script	4 hours ago